

NovaCart simplificado

Aplicación académica con **Angular/Ionic, TypeScript, Axios, PHP y MySQL de XAMPP**. El alcance se redujo a cuatro pantallas. La base utilizada es **novacart**, en el puerto **3306**.

Pantalla	Ruta	Función
Login	/login	Autenticar y abrir la página principal
Registro	/register	Crear una cuenta
Inicio	/home	CRUD de usuarios
Carrito	/cart	Agregar, consultar, modificar cantidad y quitar

Cómo abrirla

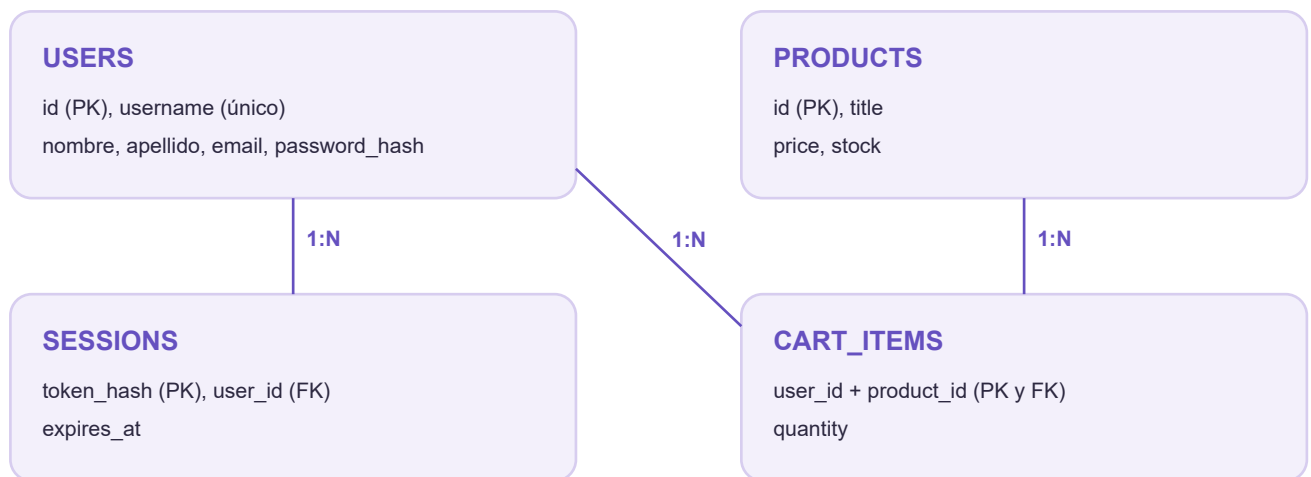
Activa Apache y MySQL en XAMPP y abre **http://localhost/novacart/**.

Cuenta demo: **emilys / emilyspass**. Para preparar otra copia:

```
npm ci
npm run db:setup
npm run deploy:xampp
```

Para desarrollar con recarga automática: **npm run dev** y **http://localhost:8100**. La conexión PHP está en **server/config.php**; las tablas, en **server/schema.sql**.

Modelo de datos



Los usuarios y carritos permanecen en MySQL. La sesión de la pestaña usa sessionStorage. La base SQLite de la versión anterior ya no se utiliza.

Login: HTML, TypeScript y API

login.page.html muestra los inputs. **login.page.ts** contiene el objeto tipado, el estado del formulario y la función asíncrona. Se conectan con `ngModel` y `ngSubmit`.

```
credentials: LoginCredentials = {
  username: '', password: ''
};

// HTML
[(ngModel)]="credentials.username"
(ngSubmit)="login()"
```

Qué ocurre al pulsar Ingresar

1. **async login(): Promise<void>** valida que haya usuario y contraseña. `Promise<void>` representa una operación asíncrona que no entrega un valor de retorno al formulario.
2. Asigna **isSubmitting = true**. Los controles quedan desactivados para evitar solicitudes repetidas. Dentro de `try`, espera la respuesta de `Axios`:

```
const { data } = await api.post<AuthResponse>(
  '/auth/login', credentials
);
saveSession(data);
await this.router.navigateByUrl('/home', {
  replaceUrl: true
});
```

3. PHP consulta MySQL y verifica la contraseña. Si es correcta, genera un token temporal. Las contraseñas están almacenadas como hash Argon2id; en `sessions` se conserva el hash del token y su vencimiento.
4. **saveSession** guarda el token, los datos públicos del usuario y `expiresAt` en `sessionStorage`. Router, inyectado por constructor, abre `home`. **replaceUrl** sustituye la entrada actual del historial.

Errores, animación y temporizadores

`catch` distingue errores HTTP con **axios.isAxiosError** y **AxiosError<ApiError>**. Si el login falla, limpia la sesión, muestra el mensaje y activa **failLogin**.

El HTML aplica la animación. El arreglo **timer** conserva los identificadores de `setTimeout`: a los 500 ms termina la animación. Los temporizadores se cancelan al reintentar y destruir la página.

finally asigna **isSubmitting = false**. Las credenciales incorrectas no abren `home`. La sesión dura una hora; al cerrar normalmente la pestaña y abrir otra se requiere login nuevamente.

Interfaces, “setear” y carrito

Setear significa asignar o establecer un valor. Por ejemplo, **this.isSubmitting = true** asigna true a una variable. Asignar un valor en memoria no lo guarda automáticamente en la base.

Una **interfaz TypeScript** describe los campos y tipos de un objeto. No crea objetos, no asigna datos ni persiste información por sí sola. La interfaz real está en `src/app/models/cart-item.model.ts`.

```
export interface CartItem {
  productId: number;
  title: string;
  price: number;
  quantity: number;
  stock: number;
}

items: CartItem[] = []; // Empieza vacío.

setCart(cart: CartResponse): void {
  this.items = cart.items; // Asigna el arreglo recibido.
  this.total = cart.total;
}
```

CartResponse contiene `items: CartItem[]` y `total: number`. Al pulsar Agregar o +/-, el componente espera **CartService.setQuantity(productId, quantity)**. Axios envía `PUT /cart/:productId` con la cantidad.

PHP valida cantidad y stock, obtiene el precio de MySQL, guarda `cart_items` y devuelve el carrito actualizado. Entonces **setCart** asigna los valores al componente. Si falla, aparece el error y se conserva el estado anterior. Cada cuenta tiene su propio carrito.

CRUD principal de usuarios

Home declara **users: User[] = []**. Al iniciar, `loadUsers(): Promise<void>` espera `UserService getUsers()` y asigna el resultado. El servicio Angular concentra las solicitudes Axios.

Acción	Solicitud de UserService
Consultar	GET /api/users
Crear	POST /api/users
Modificar	PUT /api/users/:id
Eliminar	DELETE /api/users/:id

El username es único. Al editar, una contraseña vacía conserva la actual; cambiarla revoca las sesiones. El usuario activo no puede eliminarse. Esta práctica no tiene roles ni pagos.

MCP, comprobación y uso de IA

MCP es el protocolo que permite a Codex usar herramientas para consultar MySQL. La app funciona con su API PHP; el MCP es la conexión adicional para el asistente.

App: Angular -> Axios -> Apache/PHP -> MySQL
Codex: MCP -> Node por stdio -> PHP/PDO -> MySQL

Cómo se conectó

Se instalaron **@modelcontextprotocol/sdk** y **zod** como dependencias de desarrollo. `mcp/server.mjs` registra herramientas y ejecuta `server/inspect.php`. Este archivo utiliza consultas fijas de lectura, sin aceptar SQL arbitrario.

Herramienta	Uso
<code>describe_schema</code>	Consultar tablas y columnas
<code>list_users</code>	Datos públicos de hasta 100 usuarios
<code>list_products</code>	Hasta 100 productos y precios
<code>get_cart</code>	Artículos de un usuario por ID

La configuración del proyecto está en **.codex/config.toml**:

```
[mcp_servers.novacart_mysql]
command = "node"
args = ["mcp/server.mjs"]
cwd = "C:/Fued3"
enabled = true
```

Se comprobó con **codex mcp get novacart_mysql**, **npm run test:mcp** y consultas reales desde Codex. Si se mueve la carpeta, hay que ajustar `cwd`. Para cargar cambios de configuración se vuelve a abrir el proyecto o se inicia otra sesión de Codex.

Ejemplo: “Usa `novacart_mysql` para describir las tablas” o “Consulta el carrito del usuario con id 1”. El MCP no devuelve contraseñas ni tokens y no modifica datos.

Pruebas y prompts

Resultados: **33 pruebas de Angular, 23 comprobaciones API y 5 escenarios de navegador aprobados**; lint, compilación y MCP correctos. El video muestra el cierre real del navegador y la recuperación de usuarios y carrito.

La IA ayudó a simplificar las pantallas, generar interfaces y servicios, conectar PHP/MySQL y revisar errores. Prompts principales: “3 o 4 pantallas”; “Axios, `isSubmitting`, `timer`, `failLogin` y `sessionStorage`”; “interfaz del carrito y explicar setear”; “novacart en XAMPP 3306”; “MCP para MySQL en Codex”.

Fuentes: [MCP en Codex](#); [SDK MCP](#); [PDO preparado](#). Guía completa y prompts: `docs/PROYECTO_SIMPLE.md`.